

Curso de Especialización

Ciberseguridad en las TIC

Actividad 21. CSRF.

Resumen

Autora: Marina del Pilar López Oliva

Abril 2026

IES Camp de Morvedre

Obra publicada con [Licencia Creative Commons Reconocimiento Compartir igual 4.0](https://creativecommons.org/licenses/by-sa/4.0/)



Revisión

Revisión	Fecha	Descripción
1.0	28/04/2026	Realización de la práctica.

Índice de Contenidos

1. TryHackMe CSRFv2.....	4
Tarea 4. CSRF Básico.....	4
Tarea 5. Bypass de Double Submit Cookie.....	6
Tarea 6. Bypass de SameSite Cookies.....	9
Conclusiones personales.....	11
Comprobación de la sala completada.....	11
2. responde a las siguientes cuestiones relacionadas con el apartado anterior:...	12
¿Qué tres factores importantes (fases esenciales, según TryHackme) deben cumplirse para que el ataque CSRF se produzca, además de ser vulnerable?.....	12
Explica brevemente el ataque CSRF tradicional (puedes realizar un gráfico explicativo)...	12
¿Qué idea se plantea en el laboratorio de TryHackMe para evitar esta vulnerabilidad?..	13
Aunque el método anterior funciona, todavía existen posibilidades de saltarse la protección. Indica algún método para evitar la protección que indica el laboratorio.....	13
¿Para qué se utiliza el atributo SameSite de las cookies?.....	14

1. TryHackMe CSRFv2

He completado la sala CSRFv2 de TryHackMe, la cual aborda de manera práctica el funcionamiento, explotación y defensa contra ataques de Cross-Site Request Forgery (CSRF). A continuación, se detallan los hallazgos y flags obtenidas durante el recorrido por las diferentes tareas. Hay más tareas en esta actividad, pero sólo he puesto las tareas que he considerado más importantes.

Tarea 4. CSRF Básico.

Se explota la falta de tokens CSRF en el formulario de transferencia bancaria. El atacante envía un email con una imagen oculta o un enlace malicioso que ejecuta una transferencia sin el consentimiento del usuario.

Una víctima, Josh en este caso, recibe un email falso de "MyBank" ofreciendo un premio. El email contiene un enlace a: http://mybank.thm:8080/dashboard.php?to_account=GB82MYBANK5698&amount=1000.

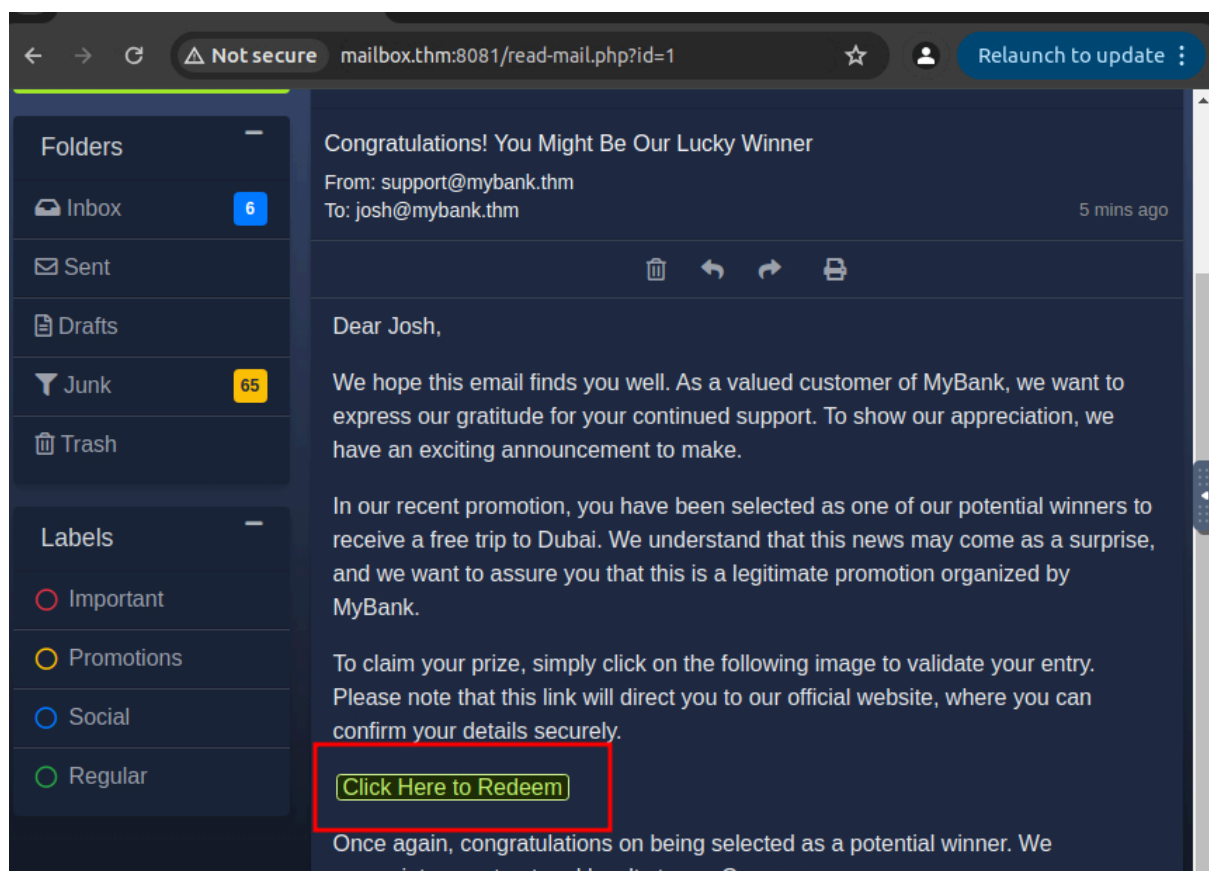


Figura 1. Correo maligno en Josh.

Al hacer clic, el navegador envía automáticamente las cookies de sesión. El servidor procesa la transferencia como legítima.

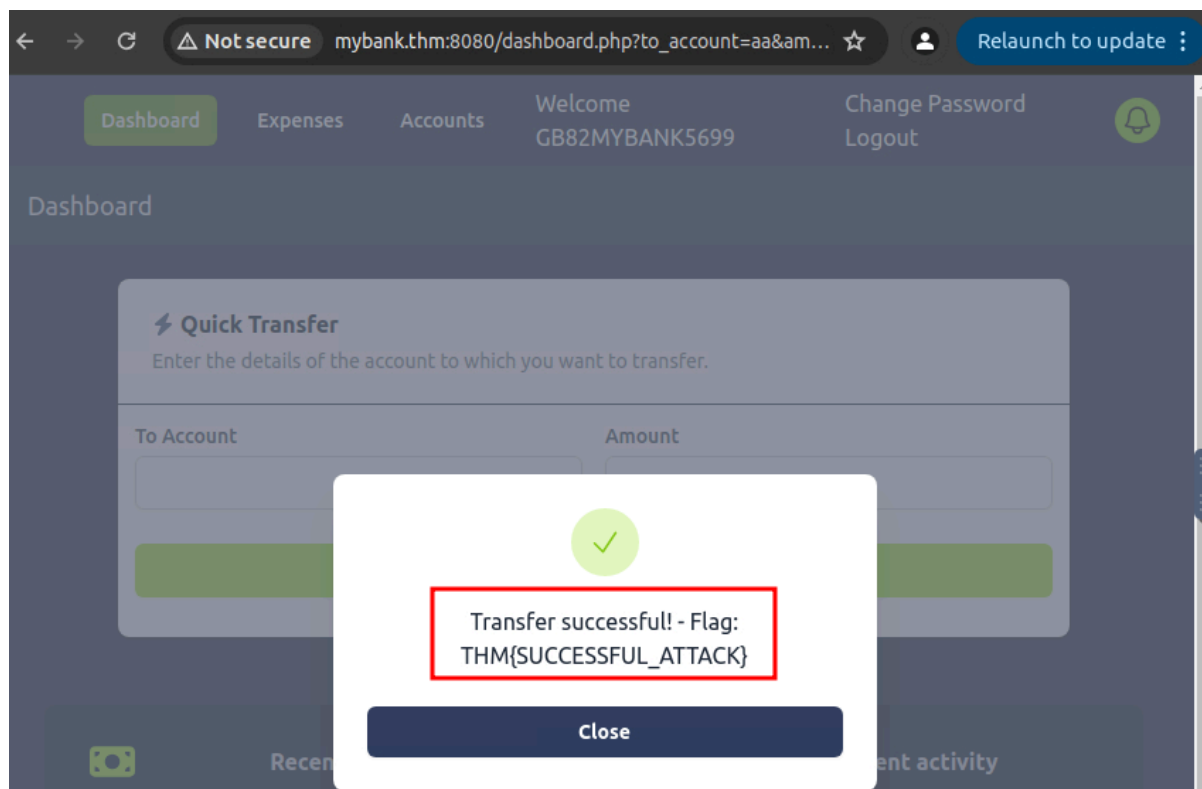


Figura 2. Flag transferencia exitosa con el correo maligno.



Figura 3. Flag ataque detectado (con token CSRF) del correo seguro.

¿La técnica de imagen oculta requiere que el atributo src apunte a una imagen legítima? No, no es necesario. Puede apuntar directamente a un endpoint malicioso.

Tarea 5. Bypass de Double Submit Cookie.

El banco implementa Double Submit Cookie: el token CSRF se envía tanto en cookie como en campo oculto. El atacante descubre que el token es predecible (Base64 del número de cuenta) y puede inyectar su propia cookie desde un subdominio controlado.

El atacante inspecciona las cookies y encuentra `csrf-token=R01Mk1ZQkFOSzu20Tk`.

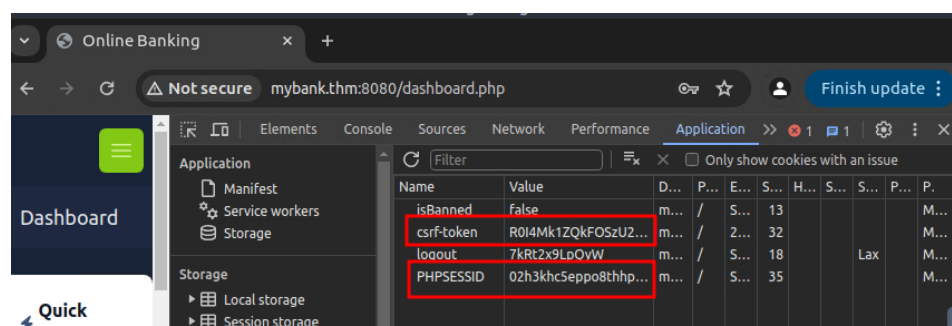


Figura 4. Cookies encontradas.

Decodifica el token con CyberChef obteniendo GB82MYBANK5699. Se debería preparar un formulario malicioso en `attacker.mybank.thm` que establezca una cookie `csrf-token` con el valor decodificado en Base64 y auto-envíe un POST a `/changepassword.php`. Josh recibe un email de "suspicious login attempt" y hace clic.

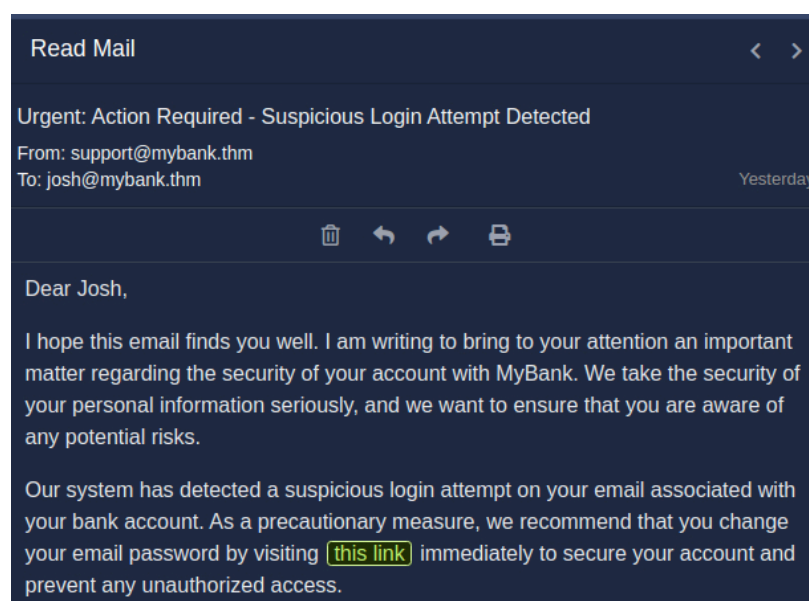


Figura 5. Correo malicioso recibido.

La contraseña se cambia sin su consentimiento.

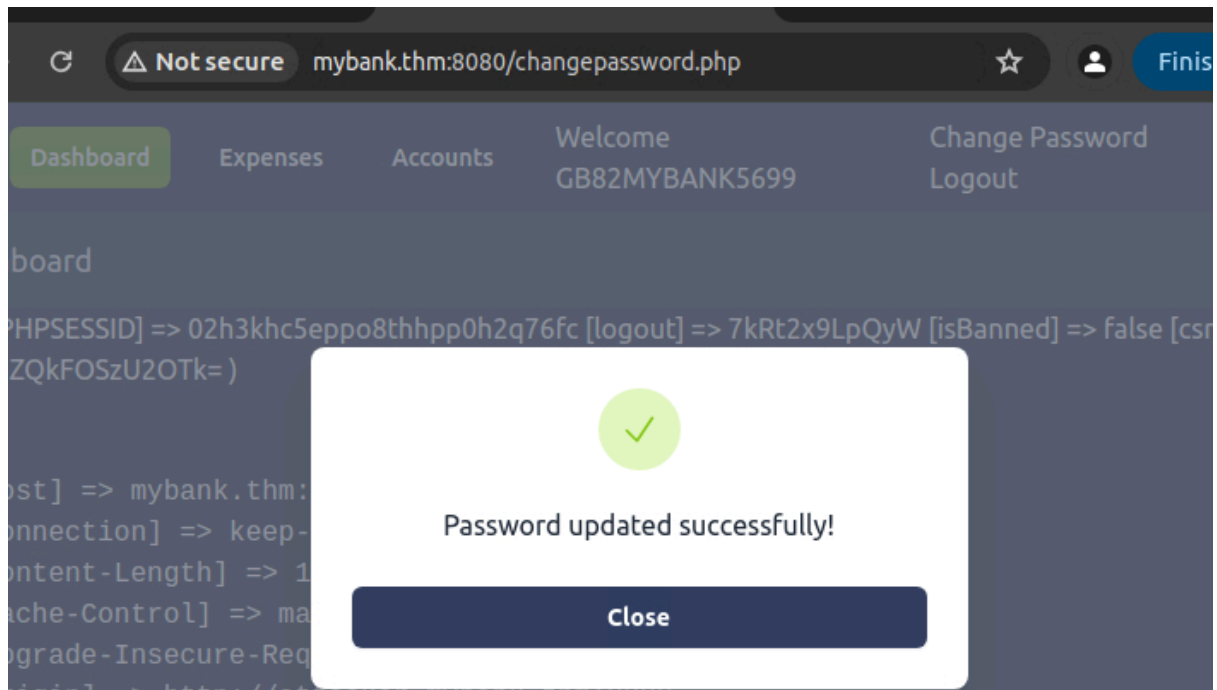


Figura 6. Contraseña cambiada.

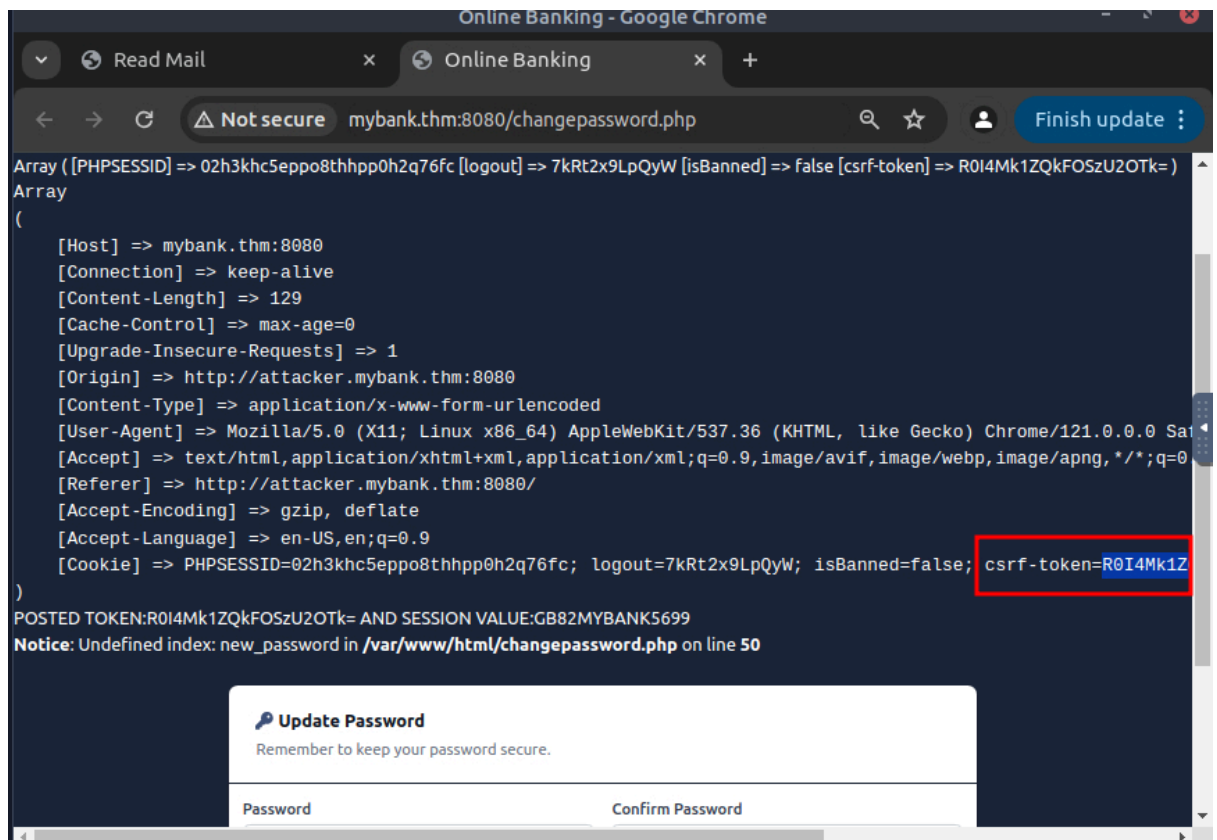


Figura 7. Token CSRF encontrado.

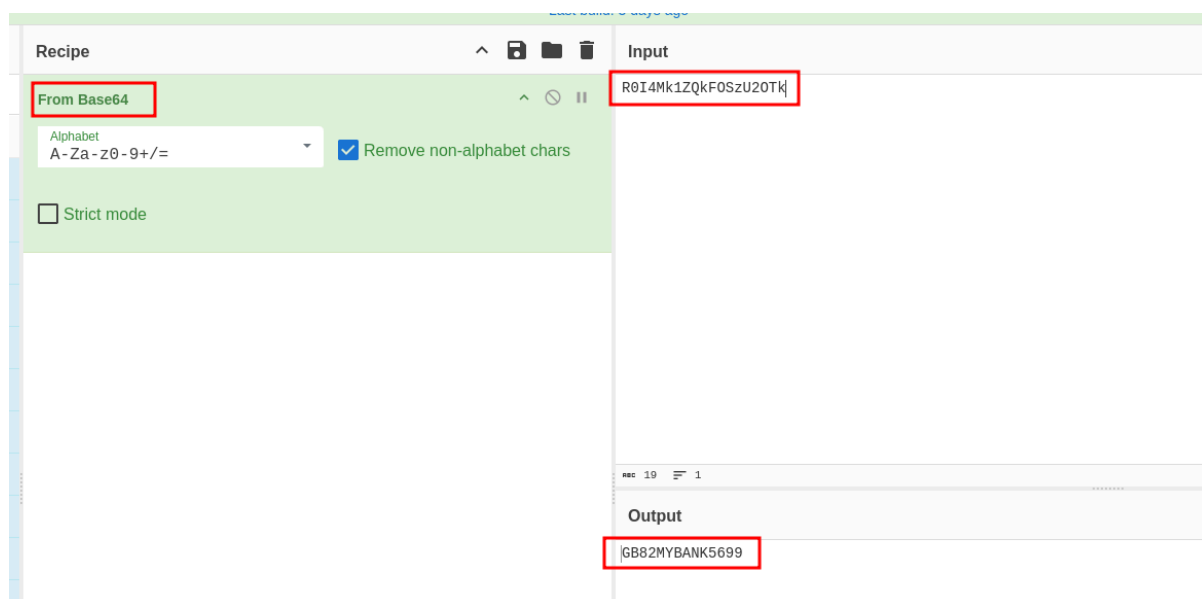


Figura 8. Decodificación del token.

¿Cuál es el valor decodificado del token CSRF para la cuenta de Josh?
GB82MYBANK5699

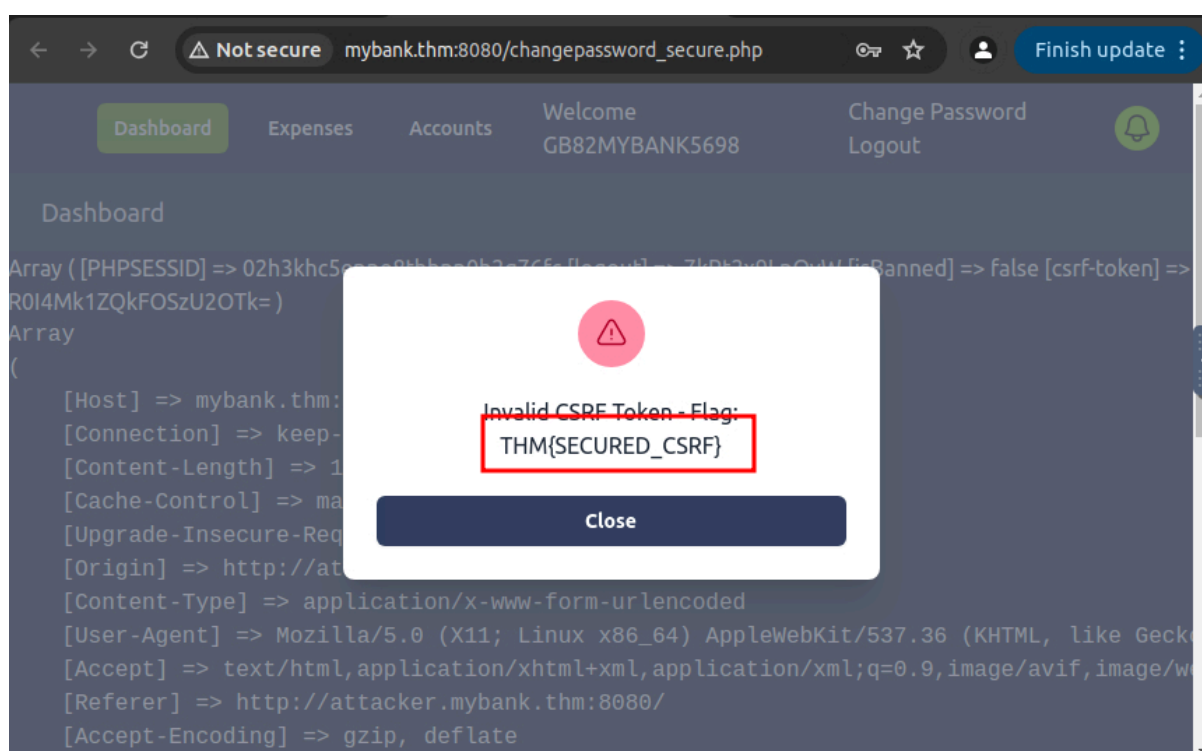


Figura 9. Flag cuando el token es aleatorio.

¿Nombre del campo oculto añadido por el banco? csrf_token

¿Es técnicamente posible secuestrar una cookie de sesión? Sí

Tarea 6. Bypass de SameSite Cookies.

La cookie logout está configurada con SameSite=Lax, lo que permite enviarla en navegaciones de nivel superior con método GET.

El atacante envía un email con un enlace a logout.php.

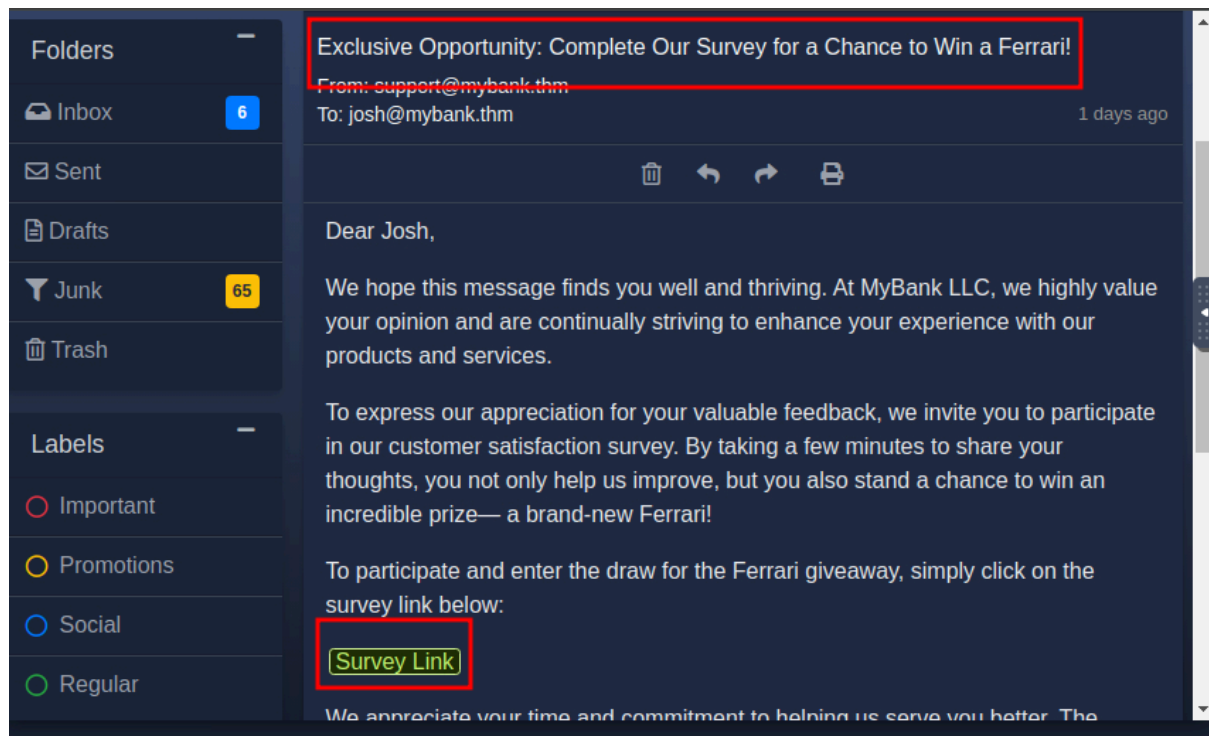


Figura 10. Email malicioso para cerrar sesión.

Josh hace clic en "Survey Link". El navegador incluye la cookie logout (por ser GET y top-level navigation). Josh es desconectado de su sesión. Al entrar de nuevo, se muestra el valor de la cookie logout.

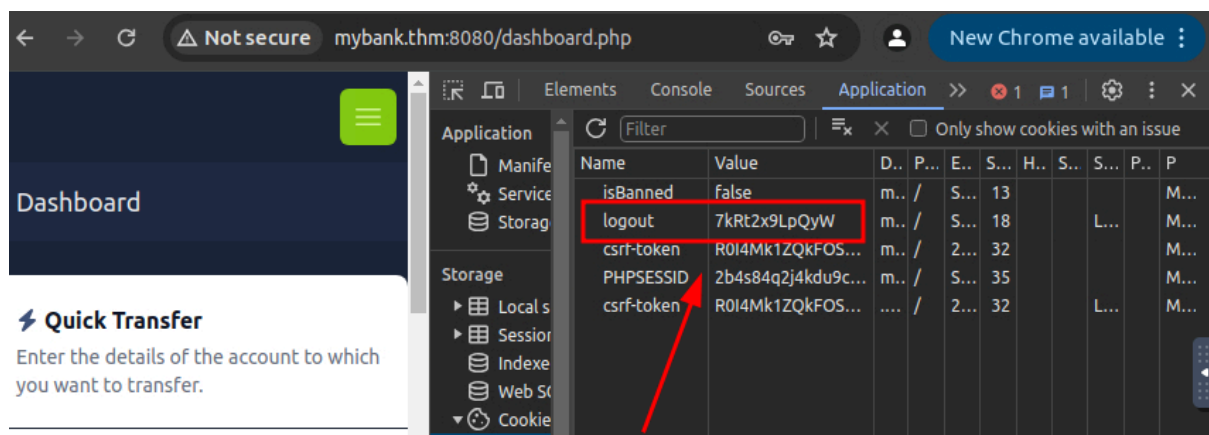


Figura 11. Valor de la cookie logout.

Name	Value	D..	P...	E..	S...	H..	S...	S
isBanned	false	m..	/	S...	13			
logout	hellothm	m..	/	S...	18			L
csrf-token	R0I4Mk1ZQkFOS...	m..	/	2...	32			
PHPSESSID	9ql37fo6me6lu9j...	m..	/	S...	35			
csrf-token	R0I4Mk1ZQkFOS...		/	2...	32			L

Figura 12. Cambio de la cookie logout a hellothm.

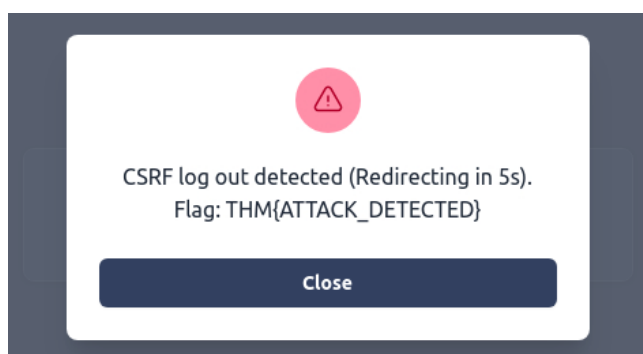


Figura 13. Flag tras cambiar logout cookie a hellothm y hacer clic.

Chrome envía cookies modificadas recientemente (menos de 2 minutos) incluso en peticiones POST cross-site. El atacante encadena un logout (que actualiza la cookie isBanned) y luego un POST para cambiar su valor.

Josh inicia sesión (cookie isBanned=false). El atacante prepara un script que debe abrir logout.php (actualiza isBanned) y enviar un POST a index.php con isBanned=true. Josh hace clic en el botón "Successful Attack". La cookie se modifica en menos de 2 minutos, por lo que Chrome la envía.



Figura 14. Flag tras actualizar isBanned=true.

Conclusiones personales.

El ataque CSRF es sencillo de ejecutar pero puede tener consecuencias graves, como transferencias no autorizadas o cambios de contraseña.

Implementar tokens CSRF no basta si estos son predecibles. En el laboratorio, el token era el número de cuenta en Base64, algo fácilmente reversible.

El atributo SameSite=Lax no es una protección absoluta, ya que permite peticiones GET en navegaciones de nivel superior, lo que puede usarse para cerrar sesiones.

La ventana de 2 minutos de Chrome para cookies modificadas recientemente permite encadenar peticiones POST que de otro modo estarían bloqueadas.

La mejor defensa contra CSRF combina tokens aleatorios, SameSite=Strict y validación de cabeceras Origin y Referer.

Comprobación de la sala completada.

A continuación se adjunta la captura de pantalla que acredita la finalización completa de la sala CSRFv2 en TryHackMe, donde se puede ver el perfil de mi usuario con la sala marcada como completada.

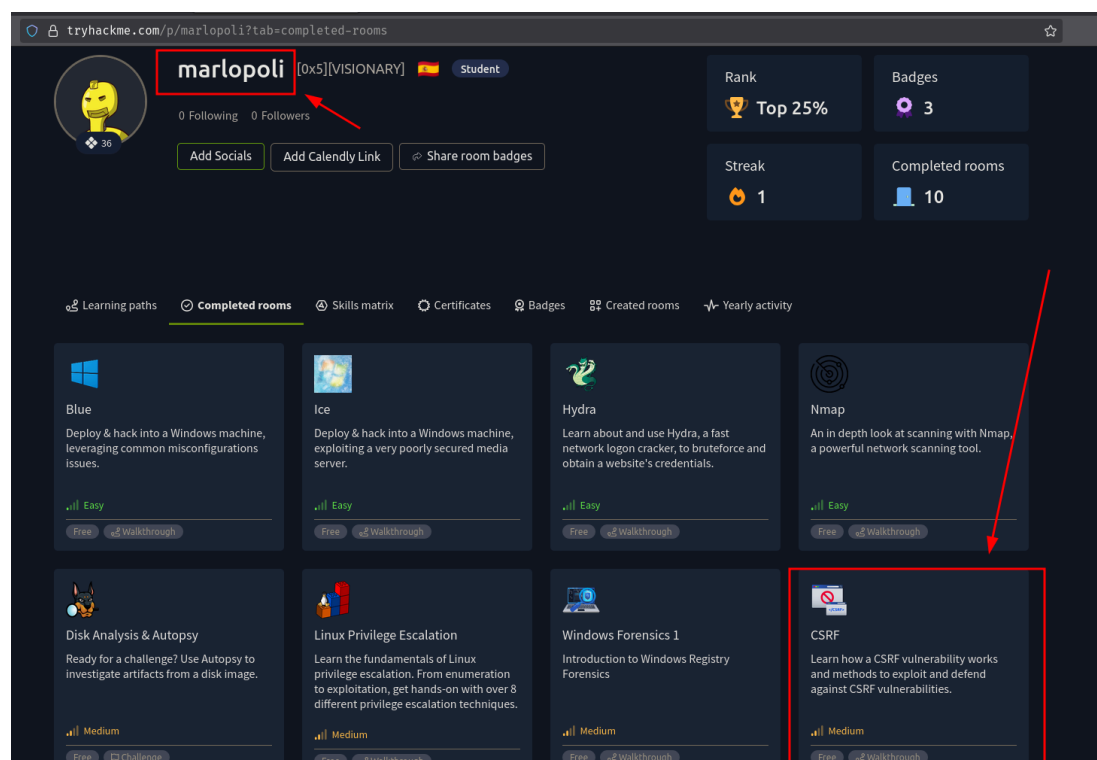


Figura 15. Sala de Tryhackme CSTFv2 completada.

2. responde a las siguientes cuestiones relacionadas con el apartado anterior:

¿Qué tres factores importantes (fases esenciales, según TryHackme) deben cumplirse para que el ataque CSRF se produzca, además de ser vulnerable?

Según lo aprendido en la sala CSRFv2 de TryHackMe, además de que la aplicación web sea vulnerable, deben cumplirse estas tres condiciones:

1. El atacante conoce la estructura de la petición vulnerable: el atacante debe conocer exactamente qué parámetros espera la aplicación para ejecutar una acción sensible (por ejemplo, transferencia de dinero, cambio de contraseña, cierre de sesión). Esto incluye conocer la URL, el método HTTP (GET o POST) y los campos necesarios.
2. La víctima tiene una sesión activa en la aplicación: la víctima debe estar autenticada en el sitio web objetivo en el momento de ejecutar el ataque. Esto permite que el navegador envíe automáticamente las cookies de sesión junto con la petición maliciosa.
3. La víctima ejecuta una acción que activa la petición maliciosa: la víctima debe interactuar con el contenido malicioso preparado por el atacante: hacer clic en un enlace, cargar una imagen oculta, enviar un formulario automático, etc. Esta acción desencadena la petición falsificada al sitio vulnerable.

Explica brevemente el ataque CSRF tradicional (puedes realizar un gráfico explicativo)

Cross-Site Request Forgery (CSRF) es una vulnerabilidad de seguridad donde un atacante engaña al navegador de una víctima para que realice una acción no deseada en un sitio web donde la víctima está autenticada.

El ataque se basa en que el navegador envía automáticamente las cookies de sesión con cada petición al dominio correspondiente. El atacante aprovecha esto para "falsificar" una petición que el servidor interpreta como legítima.

1. La víctima inicia sesión en el sitio vulnerable (ej. mybank.thm) y su navegador guarda una cookie de sesión.

2. El atacante envía un correo electrónico con un enlace malicioso o una imagen oculta.
3. La víctima abre el correo y hace clic en el enlace.
4. El navegador ejecuta la petición (GET o POST) y automáticamente incluye la cookie de sesión.
5. El servidor recibe la petición, la identifica como legítima (por la cookie válida) y ejecuta la acción.
6. El atacante logra su objetivo: transferencia de dinero, cambio de contraseña, cierre de sesión, etc.

¿Qué idea se plantea en el laboratorio de TryHackMe para evitar esta vulnerabilidad?

La principal defensa que se analiza y propone es el uso de Tokens Anti-CSRF (Tokens de Sincronización). El servidor genera un valor único, secreto e impredecible para la sesión del usuario. Este token se incluye en cada formulario o petición sensible.

Cuando el usuario envía el formulario, el servidor compara el token recibido con el que tiene guardado. Si no coinciden o el token falta, la petición se rechaza. Como un atacante externo no puede conocer este token secreto, no puede construir una petición válida.

En concreto, el laboratorio muestra cómo el banco (MyBank) solucionó el problema después del ataque. La medida clave fue: *Actualizar el algoritmo de generación de tokens para que sean únicos, aleatorios y difíciles de predecir.*

Esto se implementó en el archivo `changepassword_secure.php`, donde el token CSRF ya no es el número de cuenta en Base64 (fácilmente reversible), sino un valor generado de forma segura que el atacante no puede adivinar ni replicar.

Aunque el método anterior funciona, todavía existen posibilidades de saltarse la protección. Indica algún método para evitar la protección que indica el laboratorio

A pesar de usar tokens CSRF, el laboratorio señala que existen métodos para bypassar esta protección. Algunos de los mencionados son:

1. Inyección de cookies desde un subdominio controlado por el atacante: el atacante puede inyectar una cookie falsa desde un subdominio que

controle (ej. `attacker.mybank.thm`) hacia el dominio principal (`mybank.thm`). De esta forma, el navegador enviará la cookie falsificada junto con la petición maliciosa.

2. Explotación de tokens predecibles: si el token es generado de forma predecible (como en Base64 del número de cuenta), el atacante puede revertir el token, generar su propio token válido e inyectarlo mediante un subdominio.
3. Vulnerabilidades de XSS: si la aplicación es vulnerable a XSS, el atacante puede robar el token CSRF de la página o de las cookies y luego utilizarlo para forjar peticiones legítimas.
4. Subdomain Cookie Injection: inyectar cookies en el navegador del usuario desde un subdominio relacionado, haciendo que parezcan auténticas para el dominio principal, engañando así al sistema de protección.

¿Para qué se utiliza el atributo SameSite de las cookies?

El atributo SameSite de las cookies se utiliza para controlar cuándo y cómo se envían las cookies en peticiones de origen cruzado (cross-site). Es un mecanismo de defensa diseñado para proteger las aplicaciones web contra ataques como CSRF (Cross-Site Request Forgery) y fugas de información entre orígenes.

El atributo SameSite le indica al navegador si debe incluir o no la cookie cuando se realiza una petición desde un sitio web diferente al que la creó.

En la sala CSRFv2, el banco tenía una cookie logout configurada con SameSite=Lax. Esto permitió al atacante cerrar la sesión de Josh mediante un simple enlace GET: `<a href="https://mybank.thm:8080/logout.php" target="_blank">Survey Link!</a>`. Si la cookie hubiera estado configurada como Strict, el ataque habría fallado porque el navegador no habría enviado la cookie en una petición cross-site.